

Memory Forensics

RAM as evidence · Volatility · processes, connections, secrets

Stefanus Eko Prasetyo

Lab-7 · hands-on session

```
root@dfir:~/lab07#  
$ python2 vol.py -f dump.mem imageinfo  
Suggested Profile(s) : Win7SP1x64  
Image date and time : 2023-04-02 15:04 UTC  
# the machine still remembers
```

Today's roadmap

1

Why memory matters

what RAM preserves that disks never see

2

Acquiring memory

Dumplt, FTK Imager, Redline - and hiberfil.sys

3

Volatility basics

profiles, imageinfo, and the plugin model

4

The core plugins

pslist, pstree, netscan, envvars, cmdline

5

Hunting secrets

clipboard, command history, strings - the four hiding places

6

Challenges & working paper

upstream ransomware-gang case + two new challenges

RAM is the crime scene in motion

A memory dump is a snapshot of the machine **while it was alive** - running processes, network connections, clipboard, typed commands, even encryption keys. Disk artifacts tell you what persisted; memory tells you what was happening.

Processes

who was running, who spawned whom, who is hiding

Network

live connections and listeners - the C2 breadcrumb trail

Secrets in use

clipboard, passwords, keys - data decrypted for the moment

Volatility itself

evidence that vanishes at power-off - acquire first or lose it

Catching RAM before it vanishes

Tool	What it is
DumpIt	one-click command-line dumper for Windows (this lab's tool)
FTK Imager	the Lab 06 tool - also acquires memory
Redline	FireEye's memory acquisition + analysis agent
hiberfil.sys	offline fallback: compressed RAM image from the last boot

Order of operations in incident response: memory first, then disk, then logs - every minute of uptime changes what RAM still holds.

Meet the framework - and find its profile

```
root@dfir:~/lab07#  
$ python2 vol.py -f sample.mem imageinfo  
Suggested Profile(s) : Win7SP1x64, Win7SP0x64,  
Win2008R2SP1x64, ...  
DTB : 0x187000L    KDBG : 0xf8000283d0a0L  
Image date and time : 2023-04-02 15:04:02 UTC+0000  
  
# first suggestion is usually right:  
$ python2 vol.py -f sample.mem --profile Win7SP1x64 pslist  
# the profile = OS + arch + SP = the memory map
```

Why profiles matter

- ▶ RAM has no filesystem - just structures
- ▶ the profile tells Volatility where they live
- ▶ wrong profile = garbage or nothing
- ▶ Volatility 3 auto-detects; version 2 asks

pslist and pstree - who was alive

```
root@dfir:~/lab07#  
  
0xffffffffa8003228a10:explorer.exe    2212    1804  
. 0xffffffffa8003714f20:firefox.exe   3108    2212  
. 0xffffffffa8003938e40:ivkui.exe     3472    2212  
.. 0xffffffffa80039d1f50:powershell  3596    3472  
  
# reading the tree:  
# - odd name? unusual parent? late start time?  
# - explorer > weird.exe > powershell = classic  
#   dropper pattern
```

What to compare

- ▶ name vs its parent's expectations
- ▶ start time vs user's session
- ▶ pslist vs psscan (hidden processes)
- ▶ image path from cmdline

netscan - the wire, remembered



root@dfir:~/lab07#

Offset(P)	Proto	Local Address	Foreign Address	State	Pid	Owner
-----	-----	-----	-----	-----	----	-----
0xa34d90	TCPv4	0.0.0.0:49154	0.0.0.0:0	LISTENING	960	svchost.exe
0xb12f40	TCPv4	192.168.0.52:49188	93.184.216.34:80	CLOSE_WAIT	3108	firefox.exe
0xd01e70	TCPv4	192.168.0.52:49217	45.9.148.23:4444	ESTABLISHED	3596	powershell.exe

```
# cross-reference: netscan PID -> pslist name -> pstree parent  
# the connection outlives the moment: sockets persist in the dump
```

cmdline and envvars - how it was launched

```
root@dfir:~/lab07#  
  
$ vol.py ... cmdline -p 3472  
ivkui.exe: C:\Users\mira\AppData\Roaming\ivkui.exe  
# AppData\Roaming as an exe home = user-space persistence  
  
$ vol.py ... envvars | grep -i token  
3596 powershell APPTOKEN xK9...  
# malware stashes config where envs live;  
# legit vars are boring - odd ones stand out
```

Why these two

- ▶ cmdline gives the full invocation, not just the name
- ▶ encoded arguments (-enc) demand decoding
- ▶ envvars expose per-process config
- ▶ both survive because RAM keeps them warm

clipboard, consoles, cmdscan - the human layer

Plugin	Recovers
clipboard	whatever the user copied - CF_TEXT and formats
consoles	full console history (the visible prompt + output)
cmdscan	just the typed commands, all processes
iehistory	browser URLs and cached search queries

These four are exactly where the upstream case hides its flags: copied, searched, exported as a variable, typed as a command.

strings - the safety net under the plugins

```
root@dfir:~/lab07#  
  
$ strings -el dump.mem | grep -i flag  
ctf{...} # -el = UTF-16LE, how Windows stores text  
  
$ strings dump.mem | grep -Eo 'https?://[^\ ]+' | sort -u  
# ASCII sweep for URLs and tokens  
  
# when plugins miss it - carved areas, damaged  
# structures, unknown profile - strings still talks
```

Encodings in RAM

- ▶ ASCII: one byte per char
- ▶ UTF-16LE: two bytes, null-padded - strings -el
- ▶ wide strings are the Windows default
- ▶ grep both, trust neither alone

Why memory is the only crime scene for fileless attacks

Disk artifacts	Memory artifacts
files, MFT records, timestamps	processes, parents, command lines
registry hives	live sockets and listeners
logs (if not wiped)	clipboard, consoles, envs
cold and historical	hot and momentary

The correlation loop

- 1 imageinfo - identify the profile
- 2 pslist / pstree - spot the odd process
- 3 cmdline - see how it launched, where it lives
- 4 netscan - find its connections (match PIDs)
- 5 consoles / cmdscan / envvars / clipboard - the human layer
- 6 strings (both encodings) - verify and sweep

Every finding cites its plugin + PID.

correlation is the analysis; plugins are just the senses

The memory toolbox

Tool	Purpose
DumpIt / FTK Imager	acquire the memory dump (.raw/.mem/.vmem)
Volatility 2	the plugin framework (python2) - this lab's engine
Volatility 3	the modern rewrite - auto profile detection
strings (-el)	raw sweep, both encodings
MemLabs / Art of Memory Forensics	practice dumps and the reference book

Your challenges

Challenge A

the leader's dump

Google Drive (see README)

- ▶ Windows 7, Volatility 2
- ▶ 4 flags, 4 hiding places:
- ▶ clipboard / search / env / command
- ▶ solved together in the Deep Dive

from the original lab

Challenge B — new

triage_output.txt

files/triage_output.txt

- ▶ a full Volatility export
- ▶ one process is not like the others
- ▶ correlate tree, cmdline, sockets
- ▶ the console history talks

correlation hunt

Challenge C — new

mem_strings.bin

files/mem_strings.bin

- ▶ a raw strings-hunt blob
- ▶ four artifact types inside
- ▶ ASCII, UTF-16, base64, ROT13
- ▶ Lab 01 skills, final form

strings hunt

Flag format: `ctf{xxxx}` (Challenge A keeps the upstream `flag{xxxx}` format)

Working paper & evidence rules

1 Pre-lab first

section 1.4 of the working paper, before the session

2 Hash the dump

memory images are evidence - hash at receipt and after

3 Cite plugin + PID

every claim names the plugin and the process it came from

4 Screenshot tool output

the plugin table, not a retyped summary

5 Decode fully

-enc base64, UTF-16, URL encoding - show the chain

```
root@dfir:~/lab07#  
# evidence log entry  
[B-02] 2026-11-03 11:20  
vol.py ... nmap | findstr 3472  
45.9.148.23:8080 ivkui.exe  
-> evidence/B-02_nmap_ivkui.png
```

No plugin, no answer.

every finding cites the plugin and PID behind it

THE SERIES AT A GLANCE

01-02

the foundations: CLI, hashes, Windows artifacts

03-04

documents, macros, stego - and the web attack logs

05-06

the wire and the disk: packets, images, MFT records

07

memory: the live machine - one attacker, four labs, closed

Every artifact from Labs 4-7 belonged to the same intrusion. Today closes the case.

KEY TAKEAWAYS

01

Memory is volatile evidence

acquire first - power-off destroys the scene

02

The profile is the map

imageinfo before everything; wrong map, wrong world

03

Correlation beats any plugin

tree + cmdline + sockets + history = the story

04

Strings speak two encodings

ASCII and UTF-16LE - sweep both or miss half

The machine remembers. Now you do too.

root@dfir:~/lab07#

```
$ git clone https://github.com/stprasst/digital-forensic-lab
$ cd digital-forensic-lab/Lab\ 07/files
$ ls
triage_output.txt  mem_strings.bin
# Challenge A: download the dump (link in README)
```